

MPCSFaaS Testing Report

Authors: Akshat Singhania & Linn Oberbeck

Date: December 3, 2024

1. Executive Summary

This report documents the testing strategy and results for the MPCSFaaS (Function-as-a-Service) platform. Our test suite comprises 27 test cases covering REST API functionality, task execution, concurrent operations, error handling, and data serialization.

Test Results: 27/27 tests passed (100% pass rate)

2. Testing Approach

2.1 Testing Framework

Component	Technology
Test Framework	pytest 8.4.2
HTTP Client	requests
Serialization	dill + base64
Python Version	3.12.0

2.2 Test Categories

Our test suite is organized into two modules:

1. **test_webservice.py** - Provided integration tests (4 tests)
2. **test_extended.py** - Extended test cases (23 tests)

2.3 Test Environment

Tests require the following services to be running:

Terminal 1: `redis-server`

Terminal 2: `python3 task_dispatcher.py -m <local/pull/push>`

Terminal 3: `uvicorn server_faas:app --host 127.0.0.1 --port 8000`

Terminal 4: `pytest -v`

3. Test Suite Structure

3.1 Directory Layout

```
project/
|-- mpcsfaas/
|   |-- __init__.py
|   |-- serialize.py
|-- tests/
|   |-- __init__.py
|   |-- test_webservice.py    # Provided tests
|   |-- test_extended.py     # Extended tests
|-- pytest.ini
```

3.2 Shared Utilities

The `mpcsfaas.serialize` module provides serialization functions used across all tests:

```
def serialize(obj) -> str:
    return codecs.encode(dill.dumps(obj), "base64").decode().strip()

def deserialize(s: str):
    return dill.loads(codecs.decode(s.strip().encode(), "base64"))
```

4. Test Cases

4.1 Provided Tests (`test_webservice.py`)

Test Name	Description	Status
<code>test_fn_registration_invalid</code>	Verifies invalid payload rejection	PASSED
<code>test_fn_registration</code>	Verifies successful function registration	PASSED
<code>test_execute_fn</code>	Verifies function execution and status retrieval	PASSED
<code>test_roundtrip</code>	Verifies complete register-execute-result cycle	PASSED

4.2 Extended Tests (`test_extended.py`)

4.2.1 Registration Tests (`TestRegistration`)

Test Name	Description	Status
<code>test_register_simple_function</code>	Register a basic function	PASSED
<code>test_register_multiple_functions</code>	Register multiple unique functions	PASSED
<code>test_register_empty_payload</code>	Reject empty payload (400/500)	PASSED
<code>test_register_invalid_base64</code>	Reject invalid base64 (400/500)	PASSED

4.2.2 Execution Tests (`TestExecution`)

Test Name	Description	Status
<code>test_execute_single_arg</code>	Execute with single argument	PASSED
<code>test_execute_multiple_args</code>	Execute with multiple arguments	PASSED
<code>test_execute_with_kwargs</code>	Execute with keyword arguments	PASSED
<code>test_execute_invalid_function_id</code>	Reject non-existent function (404)	PASSED
<code>test_execute_malformed_function_id</code>	Reject malformed UUID (400)	PASSED

4.2.3 Status Tests (`TestStatus`)

Test Name	Description.	Status
<code>test_status_endpoint</code>	Verify <code>/status/{task_id}</code> endpoint	PASSED
<code>test_result_endpoint</code>	Verify <code>/result/{task_id}</code> endpoint	PASSED
<code>test_invalid_task_id</code>	Reject non-existent task (404)	PASSED
<code>test_malformed_task_id</code>	Reject malformed UUID (400)	PASSED

4.2.4 Concurrent Tests (`TestConcurrent`)

Test Name	Description	Status
<code>test_concurrent_same_function</code>	10 concurrent calls to same function	PASSED
<code>test_concurrent_different_functions</code>	Concurrent calls to different functions	PASSED

Test Name	Description	Status
test_many_concurrent_tasks	50 concurrent task submissions	PASSED

4.2.5 Error Handling Tests (TestErrorHandling)

Test Name	Description	Status
test_function_exception	Verify exception handling returns FAILED status	PASSED

4.2.6 Data Type Tests (TestDataTypes)

Test Name	Description	Status
test_string_input	Serialize/deserialize string data	PASSED
test_list_input	Serialize/deserialize list data	PASSED
test_dict_input	Serialize/deserialize dictionary data	PASSED
test_nested_data	Serialize/deserialize nested structures	PASSED

4.2.7 API Tests (TestAPI)

Test Name	Description	Status
test_root_endpoint	Verify root endpoint returns service status	PASSED
test_content_type_json	Verify JSON content type handling	PASSED

5. Test Coverage Analysis

5.1 API Endpoint Coverage

Endpoint	Method	Tests
/	GET	1
/register_function	POST	6
/execute_function	POST	8
/status/{task_id}	GET	3
/result/{task_id}	GET	4

5.2 Functional Coverage

Category	Coverage
Function Registration	Covered
Function Execution	Covered
Task Status Retrieval	Covered
Result Retrieval	Covered
Error Handling	Covered
Input Validation	Covered
Concurrent Execution	Covered
Data Serialization	Covered

5.3 Status Code Coverage

Status Code	Meaning	Tested
200	Success	Yes
400	Bad Request	Yes
404	Not Found	Yes
500	Server Error	Yes

6. Test Results Summary

6.1 Overall Results

```
===== test session starts =====
platform darwin -- Python 3.12.0, pytest-8.4.2, pluggy-1.6.0
collected 27 items

tests/test_extended.py::TestRegistration::test_register_simple_function PASSED
tests/test_extended.py::TestRegistration::test_register_multiple_functions PASSED
tests/test_extended.py::TestRegistration::test_register_empty_payload PASSED
tests/test_extended.py::TestRegistration::test_register_invalid_base64 PASSED
tests/test_extended.py::TestExecution::test_execute_single_arg PASSED
tests/test_extended.py::TestExecution::test_execute_multiple_args PASSED
tests/test_extended.py::TestExecution::test_execute_with_kwargs PASSED
tests/test_extended.py::TestExecution::test_execute_invalid_function_id PASSED
tests/test_extended.py::TestExecution::test_execute_malformed_function_id PASSED
tests/test_extended.py::TestStatus::test_status_endpoint PASSED
tests/test_extended.py::TestStatus::test_result_endpoint PASSED
tests/test_extended.py::TestStatus::test_invalid_task_id PASSED
tests/test_extended.py::TestStatus::test_malformed_task_id PASSED
tests/test_extended.py::TestConcurrent::test_concurrent_same_function PASSED
tests/test_extended.py::TestConcurrent::test_concurrent_different_functions PASSED
tests/test_extended.py::TestConcurrent::test_many_concurrent_tasks PASSED
tests/test_extended.py::TestErrorHandling::test_function_exception PASSED
tests/test_extended.py::TestDataTypes::test_string_input PASSED
tests/test_extended.py::TestDataTypes::test_list_input PASSED
tests/test_extended.py::TestDataTypes::test_dict_input PASSED
tests/test_extended.py::TestDataTypes::test_nested_data PASSED
tests/test_extended.py::TestAPI::test_root_endpoint PASSED
tests/test_extended.py::TestAPI::test_content_type_json PASSED
tests/test_webservice.py::test_fn_registration_invalid PASSED
tests/test_webservice.py::test_fn_registration PASSED
tests/test_webservice.py::test_execute_fn PASSED
tests/test_webservice.py::test_roundtrip PASSED

===== 27 passed in 0.73s =====
```

6.2 Statistics

Metric	Value
Total Tests	27
Passed	27
Failed	0
Pass Rate	100%
Execution Time for local Mode	0.73s
Execution Time for pull Mode	2.79s
Execution Time for push Mode	2.12s

7. Test Functions Used

The following test functions were used across the test suite:

```
def double(x):
    return x * 2

def add(x, y):
    return x + y

def multiply(x, y):
    return x * y

def square(x):
    return x * x

def identity(x):
    return x

def sleep_task(duration):
    import time
    time.sleep(duration)
    return f"slept {duration}s"

def compute_sum(n):
    total = 0
    for i in range(n):
        total += i
    return total

def failing_function(x):
    raise ValueError(f"Intentional error with input {x}")
```

8. Running the Tests

8.1 Prerequisites

Install dependencies

pip install pytest requests dill

Start services

redis-server

python3 task_dispatcher.py -m <local/pull/push>

uvicorn server_faas:app --host 127.0.0.1 --port 8000

Terminal 1

Terminal 2

Terminal 3

8.2 Test Commands

Run all tests

pytest -v

Run only provided tests

pytest tests/test_webservice.py -v

Run only extended tests

pytest tests/test_extended.py -v

Run specific test class

pytest tests/test_extended.py::TestConcurrent -v

```
# Run specific test
pytest tests/test_extended.py::TestExecution::test_execute_single_arg -v

# Run with coverage (requires pytest-cov)
pytest --cov=mpcsfaas -v
```

9. Known Limitations

1. **Integration Tests Only:** Tests require running services
 2. **Single Mode Testing:** Tests run against one dispatcher mode at a time; switching modes requires service restart
 3. **Fixed Timeout:** Tests use 30-second timeout which may not be suitable for all environments
-

10. Future Test Improvements

1. **Add Unit Tests:** Test individual components in isolation
 2. **Add Load Tests:** Test system behavior under heavy load
-

11. Conclusion

The MPCSFaaS test suite provides comprehensive coverage of the REST API functionality with 27 test cases achieving a 100% pass rate. The tests verify core functionality including function registration, task execution, status tracking, concurrent operations, error handling, and data serialization.

The test suite successfully validates that the implementation meets the specified interface requirements and handles both valid operations and error conditions appropriately.